AI Lab Assignment 1

Code Explanation: Shortest Path Using BFS, UCS and A*

1. Aim and problem statement

Find a route from **Pune** (start) to **Ratnagiri** (goal) using three search algorithms: Breadth-First Search (BFS), Uniform Cost Search (UCS), and A* search. The graph contains eight cities. Each road has a distance (edge cost), and A* also uses a heuristic estimate of the remaining distance to the goal.

2. Understand the graph

The graph is stored as a Python dictionary of dictionaries, which is an adjacency-list representation. For example:

```
"pune": {"mumbai": 140}
```

This means Pune is connected to Mumbai and the road distance is 140 km. The city names are nodes, roads are edges, and numbers are edge costs. Since roads are listed in both directions (for example, Pune–Mumbai and Mumbai–Pune), the graph is undirected.

The heuristic dictionary h estimates each city's remaining distance to Ratnagiri. The goal has heuristic 0 because no distance remains once we reach it.

3. BFS — Breadth-First Search

Type: Uninformed search. BFS explores nodes level by level using a FIFO queue (First In, First Out).

```
queue = [[start, [start]]]
visited = [start]
```

- **queue:** stores the current city and the path used to reach it. Initially it contains Pune and the path [Pune].
- **visited:** prevents a city from being added repeatedly.
- **queue.pop(0):** removes the first queue entry, following FIFO order.
- **if city == goal:** returns the path when Ratnagiri is reached.
- **path + [neighbour]:** creates a new path with the neighbour appended.

Key limitation: BFS finds a path with the fewest edges, but it does not account for different road distances. In this code it returns only the route, not its total distance.

4. UCS — Uniform Cost Search

Type: Uninformed search. UCS expands the available path with the lowest accumulated road cost.

```
queue = [[0, start, [start]]]
```

Each queue entry contains [total_cost, city, path]. The journey starts at Pune with cost 0.

```
new_cost = cost + graph[city][neighbour]
```

This adds the road distance to the distance already travelled. The code scans the list to find the entry with the smallest cost, removes it, and expands its neighbours. If the city is already in `visited`, it skips it. When the current city is Ratnagiri, it returns both the path and cost.

Key point: UCS considers road distances, so with non-negative edge costs it can find a minimum-cost path.

5. A* search

Type: Informed search. A* uses both the actual distance travelled and a heuristic estimate.

Formula: f(n) = g(n) + h(n)

- **g(n):** actual distance from Pune to the current city.
- **h(n):** estimated remaining distance from the current city to Ratnagiri.
- **f(n):** estimated total cost through the current city.

```
queue = [[h[start], 0, start, [start]]]
best = {start: 0}
```

The A* queue stores [f_cost, actual_cost, city, path]. The `best` dictionary records the lowest actual cost found so far for each city. For every neighbour, the code calculates a new actual cost. If it is a better route, it updates `best`, calculates `new_f = new_cost + h[neighbour]`, and adds the improved entry to the queue. The entry with the smallest f-value is explored first.

Heuristic caution: The supplied values are illustrative, not verified straight-line distances. Some may overestimate the cheapest remaining road cost, so these values do not guarantee A* optimality for every graph.

6. Output and result

```
BFS: pune->mumbai->sind->ratnagiri
UCS: pune->mumbai->sind->ratnagiri | Distance: 470 km
A*: pune->mumbai->sind->ratnagiri | Distance: 470 km
```

The route distance is 140 + 220 + 110 = **470 km**. All three algorithms return the same route in this example. BFS does not calculate the distance; UCS and A* return the path and actual cost.

7. Comparison

Feature	BFS	UCS	A*
Search type	Uninformed	Uninformed	Informed
Selection rule	Level by level	Lowest actual cost	Lowest f = g + h
Uses road cost?	No	Yes	Yes
Uses heuristic?	No	No	Yes
Returns distance?	No	Yes	Yes

8. Short viva explanation

“Ma'am, I represented eight cities as nodes and roads as weighted edges. BFS explores level by level without considering road distances. UCS selects the path with the lowest accumulated cost. A* uses f(n) = g(n) + h(n), combining actual cost with an estimated remaining distance. In this example, all three return Pune → Mumbai → Sind → Ratnagiri; UCS and A* calculate the total distance as 470 km.”

9. Likely viva questions

Q. What is a heuristic?

A. An estimate of the remaining cost from a city to the goal.

Q. Why is h(Ratnagiri) = 0?

A. Because the goal has been reached, so the remaining distance is zero.

Q. Why is BFS not always shortest by distance?

A. It minimises the number of edges, not the sum of road distances.

Q. Difference between UCS and A*?

A. UCS uses accumulated cost g(n); A* uses g(n) + h(n).

Q. What does best store in A*?

A. The lowest actual cost found so far for each city.

Q. Is the heuristic fully correct for the statement?

A. Not yet; use verified straight-line distances to Ratnagiri rather than illustrative estimates.